UNIVERSITY OF TRIESTE

Department of Mathematics, Informatics and Geosciences

Master's Degree in Data Science and Artificial Intelligence

Curriculum: Foundations of Artificial Intelligence and Machine Learning

Unsupervised State Bucketing for Mixture of Experts in Resource-Constrained Chess Engines

Graduation session: 23 October 2026

Candidate
Omar Cusma Fait
Student ID SM3800018

Supervisor
Prof. Alessio Ansuini

Internship supervisor
Prof. Marco Zennaro
ICTP

Academic Year 2025/2026

Lorem ipsum
dolor sit amet.
— Cicero – *De finibus bonorum et malorum* —

Ad maiora!

Abstract

In the context of board games, a common approach to increasing model performance is to partition the state space and allocate distinct experts to each region. While this is the central idea behind Mixture of Experts (MoE) architectures, the bucketing is almost always defined manually, requiring domain-specific expertise, and yielding potentially suboptimal partitions.

While unsupervised bucketing using hidden layer activations has been explored, this approach may not yield partitions that *maximise expert specialisation*. Activations capture what the model *knows*, not what it *needs* to adjust. In this work, we ask whether we can use the model's learning dynamics — the sample-gradients — to create a partition that is more effective. Our hypothesis is that clustering gradients (which encode the direction of desired weight updates) will produce buckets that are inherently better suited for expert fine-tuning, leading to more specialised experts.

We propose a novel unsupervised bucketing method based on sample-gradients — the gradients of the loss with respect to the head parameters of a base model. These vectors encode the direction in weight space that would improve the model's prediction for each individual data point. We apply the method to chess, using a standard NNUE (Efficiently Updatable Neural Network) as the base model trained on 5 million positions labelled with outcome probabilities (WDL) by a strong value function (Lc0).

For each position, we compute the normalised sample-gradient with respect to the head parameters, and apply a clustering algorithm to define the buckets. We then train a lightweight linear dispatcher to predict the bucket from the L1 activations, enabling fast, deterministic routing at inference time. Finally, we fine-tune the expert heads on each bucket, starting from the base model, while keeping the L1 weights frozen. This yields a set of fast, specialised experts, each adapted to a distinct region of the state space, without requiring handcrafted bucketing.

Contents

Chapter 1

Introduction

Motivate the work, state the research questions, list contributions, and outline the rest of the thesis.

1.1 Motivation

Chess engines on embedded devices (Wio Terminal: 192 KB RAM, 500 KB flash) need an efficient evaluation function such as NNUE. Handcrafted bucketing (piece count, queen presence) is heuristic and may not capture the true structure of the state space.

1.2 Problem Statement

How can we automatically learn a partition of the state space that maximises expert specialisation, and how can we do this without human supervision?

1.3 Contributions

(i) An unsupervised bucketing method based on sample gradients (gradients with respect to head parameters). (ii) A lightweight dispatcher that routes positions to the right expert at inference time. (iii) A Mixture of Experts NNUE architecture for resource-constrained devices. (iv) Empirical validation on chess against a single-head baseline.

1.4 Thesis Outline

Roadmap: Chapter 2 reviews engines, NNUE, MoE, sample gradients, and bucketing. Chapter 3 presents the five-step unsupervised bucketing pipeline. Chapter 4 describes the chess/NNUE instantiation. Chapter 5 reports experiments. Chapter 6 discusses results and limitations. Chapter 7 concludes and sketches future work.

Chapter 2

Background and Related Work

Give the reader the minimum chess, NNUE, MoE, and gradient background needed to follow the method, then situate the thesis against related work.

2.1 Chess Engines and Evaluation Functions

Classical evaluation (material, piece-square tables, mobility, king safety). NNUE: sparse, accumulator-based efficiently updatable networks. Search: alpha-beta, iterative deepening, move ordering.

2.2 Mixture of Experts

Multiple expert networks, each specialising in a sub-domain. Routing: learned vs. fixed. Applications in vision, NLP, and reinforcement learning.

2.3 Sample Gradients

Define sample (per-example) gradients with respect to the output-head parameters, and why they are a useful similarity signal for partitioning the state space. Related literature on gradient clustering still needs a closer survey.

2.4 Bucketing in NNUE

Default techniques: piece count, king location, piece presence (queen, bishop pair, etc.). Limitations: handcrafted rules may not align with what the model actually needs.

2.5 Related Work

Efficiency: EfficientNet, pruning, quantization. Distillation and teacher–student training. Reinforcement learning from self-play (AlphaZero, Lc0). Expand the survey of work related to sample gradients.

Chapter 3

Method: Unsupervised Bucketing via Sample Gradients

This is the core contribution. Present the pipeline independently of chess, with notation, then specialise only where needed.

3.1 Overview

Five-step pipeline: (1) train a base model; (2) compute sample gradients; (3) cluster sample gradients; (4) train a dispatcher; (5) fine-tune expert heads.

3.2 Notation and Definitions

State space $\mathcal{S}$, model f_w, loss $\mathcal{L}$, target $\hat{v}$. Sample gradient $\delta_i = \nabla_{w^{(\text{head})}} \mathcal{L}(f_w(s_i), \hat{v}_i)$. Dispatcher $g_\phi(h) = \text{softmax}(W_\phi h)$.

3.3 Step 1: Train the Base Model

Architecture: L1 (dual point-of-view, shared), L2, output head. Training: soft cross-entropy on WDL labels from Lc0.

3.4 Step 2: Compute Sample Gradients

Per-sample gradients with respect to head parameters. Normalisation (L2 or standardisation). Storage and computational cost.

3.5 Step 3: Cluster Sample Gradients

K-Means, or alternatives such as DBSCAN or Density Peak (some methods choose B automatically). Choosing the number of clusters B. Validation: inertia, silhouette score, cluster interpretability.

3.6 Step 4: Train the Dispatcher

Input: L1 activations h. Output: bucket id b. Architecture: linear layer or small MLP. Loss: cross-entropy.

3.7 Step 5: Fine-Tune Expert Heads

Freeze L1, fine-tune one head per bucket. Result: B specialised experts.

3.8 Generalisation Beyond Chess

The method only needs a state space, a model, a loss, and a target (e.g. expected reward). Candidates: other games (Go, Shogi), robotics, and settings with a world model.

Chapter 4

Implementation: NNUE and MoE for Chess

Instantiate the method on chess: data, architecture, training details, and how the MoE NNUE is plugged into Cfish on the target device.

4.1 Dataset and Teacher

Lichess PGNs → FEN positions → Lc0 WDL labels (depth 1). About 5M positions. Preprocessing: 844-dim feature encoder (716 base + 128 tactical), dual point-of-view.

4.2 Base NNUE Architecture

L1: $844 \to 64$ *(shared, dual-POV). L2:* $128 \to 128$. *Output:* $128 \to 3$ *WDL logits. Activations: CReLU (L1), ReLU (L2), softmax (output). Training: soft cross-entropy, Adam, 100 epochs.*

4.3 Sample Gradient Computation

Implementation: `torch.autograd.grad`*, per-sample gradients. Storage:* `.npy` *arrays, optionally* `.parquet` *for metadata.*

4.4 Clustering

Mini-Batch K-Means for scale; Density Peak Clustering; other algorithms to be decided. Working choice $B = 8$ *(to match Stockfish-style bucketing; to be checked). Visualisation: t-SNE, PCA.*

4.5 Dispatcher Training

Input: L1 activations concatenated as [own‖opp] *(own side alone may suffice). Architecture: linear* $2W \to B$ *(or* $W \to B$*). Training: cross-entropy, Adam, 20 epochs.*

4.6 Expert Fine-Tuning

One sweep per bucket vs. full training: still to decide. Validation: per-bucket test cross-entropy.

4.7 Final MoE Model

Shared L1 + dispatcher + expert heads. Inference: L1 $\to$ *dispatcher* $\to$ *selected expert* $\to$ *output.*

4.8 Integration with Cfish

Replace the `evaluate()` *hook with the MoE NNUE. Quantization: int8 weights, int16 accumulators, int32 MAC. Memory: sparse L1 (70–80% pruning); dispatcher and heads in flash.*

Chapter 5

Experiments and Results

Core empirical chapter. Protocol, metrics, then results for the base model, clustering/dispatcher, MoE, playing strength, ablations, and visualisations. Several numbers below are targets or current snapshots, not final.

5.1 Experimental Setup

Split: train 2.4M, test 31k. Hardware: CPU for training, Wio Terminal for inference. Baselines: single-head NNUE; dense FFNN (256×256); handcrafted bucketing (piece count + queen presence).

5.2 Evaluation Metrics

Test cross-entropy (CE). MAE on expected reward. Dispatcher accuracy. Per-bucket test CE vs. the base head. Playing strength (ACPL / Elo) via Stockfish analysis.

5.3 Results: Base Model

Snapshot: test CE 0.71 *after 50 epochs; MAE* 0.25. *Compare with FFNN (*0.74 *CE, 499k parameters).*

5.4 Results: Clustering and Dispatcher

Cluster quality (silhouette). Target dispatcher accuracy $> 70\%$. *Interpret clusters (endgame vs. middlegame, tactical vs. positional).*

5.5 Results: MoE Model

Test CE still TBD (goal: below the 0.71 *baseline). Per-bucket improvement over the base head. Ablation on* $B \in \{4, 8, 16\}$.

5.6 Results: Playing Strength

To compute: ACPL / Elo vs. baseline. Cfish on Wio: nps, depth, move time.

5.7 Ablation Studies

Optional: number of clusters B*; dispatcher architecture (linear vs. MLP); fine-tuning duration (1 vs. 5 epochs).*

5.8 Visualisation

t-SNE / PCA of sample gradients. Dispatcher decision boundaries. Expert activation distribution.

Chapter 6

Discussion

Interpret the empirical findings; do not introduce new experiments. Emphasise that unsupervised bucketing via sample gradients is the claimed novelty, and chess is the validation domain.

6.1 Interpretation of Results

What the numbers mean for specialisation, evaluation quality, and on-device cost. Which buckets are semantically meaningful.

6.2 Limitations

Likely limits: clustering quality, dispatcher errors, memory/compute on the Wio Terminal, sensitivity to B, dependence on the teacher (Lc0) and on a single game.

6.3 Generalisation

Can the method transfer to other domains? What must change (features, loss, target, hardware constraints)?

6.4 Comparison with Related Work

Vs. handcrafted NNUE bucketing. Vs. learned routing in other MoE systems.

Chapter 7

Conclusion and Future Work

Close the thesis: restate contributions, list follow-ups, and end with a short take-away. Target length of the full document: about 40–50 pages, paper-style.

7.1 Summary of Contributions

Recap the unsupervised gradient-based bucketing method, the lightweight MoE NNUE, and the chess-engine validation.

7.2 Future Work

Possible directions: better clustering / automatic B; stronger dispatcher; other games or world-model settings; tighter on-device integration and playing-strength evaluation.

7.3 Closing Remarks

One or two sentences: automatic partitions of the state space can replace hand-crafted buckets when evaluation must stay tiny.

Appendix A

Supplementary Material

Place extra material that would interrupt the main argument: feature-encoder tables, extra plots, hyperparameters, dataset statistics, and any clustering diagnostics. Content still to be decided.

Bibliography

[1] Michel Goossens, Frank Mittelbach, and Alexander Samarin. *The LaTeX Companion*. Reading, Massachusetts: Addison-Wesley, 1993.

Ei fu. Siccome immobile,
Dato il mortal sospiro,
Stette la spoglia immemore
Orba di tanto spiro

Alessandro Manzoni – *Il Cinque Maggio*